

CS1910

Dictionaries

JUPYTER NOTEBOOK

- <https://colab.research.google.com/github/cdspower-upei/2026W-CS1910-Students/blob/main/L13/CS1910-Dictionary-Examples.ipynb>



DICTIONARY

- A collection of data (a list is also a collection of data)
- Does not use numbered indices
- Uses **keys** to access **values**
- Similar to a real-world dictionary
 - A word is associated with its definition
- Key-value pairs: information stored as pairs, a unique key and its associated value
- Mutable
 - You can add, change or remove items after creating a dictionary

DICTIONARY

- The key in a python dictionary can be any immutable type (int, string, tuple, etc)
- The value can be anything (another string, a list, an int, another dictionary)
- Example:
 - Key: student id as a string; Value: a list of strings representing enrolled course codes
 - Key: course code as a string; Value: the number of students in the class

```
1  # Students and courses
2  student_courses = {"00100" : ["CS1910", "MATH1910", "STAT1910"]}
3  print(student_courses) # {'00100': ['CS1910', 'MATH1910', 'STAT1910']}
4  course_students = {"CS1910" : 20,
5                      "MATH1910" : 30,
6                      "STAT1910" : 40
7                      }
8  print(course_students) #{'CS1910': 20, 'MATH1910': 30, 'STAT1910': 40}
```

CREATING A DICTIONARY

```
1 #empty dictionary
2 my_dict = { }
3 #empty dictionary
4 my_dict2 = dict()
5 #populated dictionary associating names with email addresses
6 contacts = { "Clark Kent" : "ckent@dailyplanet.com",
7              "Superman"   : "ckent@dailyplanet.com" }
```

Must be unique and immutable



Values can be any data type and
duplicates are allowed



DICTIONARY EXAMPLE

- Values can be of different types:

```
1 contact = {"name" : "Clark Kent",  
2           "age" : 33,  
3           "address" : "Metropolis" }
```

{'name': 'Clark Kent', 'age': 33, 'address': 'Metropolis'}

- When creating the key always takes the last value it was set to even with type change

```
1 contacts = {"name" : "Clark Kent",  
2            "age" : 33,  
3            "address" : "Metropolis",  
4            "name" : "Bruce Wayne",  
5            "age" : "Unknown",  
6            "address" : "Gotham" }
```

{'name': 'Bruce Wayne', 'age': 'Unknown', 'address': 'Gotham'}

ACCESSING VALUES

```
01 # Using Dictionaries
02 contact = {"name" : "Clark Kent",
03            "age" : 33,
04            "address" : "Metropolis" }
05 print(contact) # {'name': 'Clark Kent', 'age': 33, 'address': 'Metropolis'}
06 # Get value associated with key "age"
07 print(contact["age"]) # 33
08 print(contacts["secret_identity"]) # Produces KeyError
09 # Update key
10 contact["name"] = "Superman"
11 print(contact) # {'name': 'Superman', 'age': 33, 'address': 'Metropolis'}
12 # Add a new Key
13 contact["secret_identity"] = "Clark Kent"
14 print(contact["secret_identity"]) # Clark Kent
```

REUSING KEYS

If you accidentally reuse keys during creation (or later in a program) the value will simply be overwritten

```
01 # Duplicate Keys - Results in Overwrite
02 contacts = {"name" : "Clark Kent",
03             "age" : 33,
04             "address" : "Metropolis",
05             "name" : "Bruce Wayne",
06             "age" : "Unknown",
07             "address" : "Gotham" }
08
09 # Reusing keys in creation results in overwriting values
10 print(contacts)

#{'name': 'Bruce Wayne', 'age': 'Unknown', 'address': 'Gotham'}
```


LIST OF DICTIONARIES

```
01 #List of Dictionaries
02 contacts = [ { "name" : "Clark Kent",
03               "age" : 33,
04               "address" : "Metropolis" },
05               { "name" : "Bruce Wayne",
06               "age" : "Unknown",
07               "address" : "Gotham" }
08           ]
09 print(len(contacts)) # Length of list contacts
10 print(len(contacts[0])) # Number of items in dictionary
11 print(contacts[0]) # dictionary at 0
12 print(contacts[1]) # dictionary at 1
```

- In the above case, we are storing repeated dictionaries within a list – that allows us to store sets of values in a predictable way

- Output:

2

3

{'name': 'Clark Kent', 'age': 33, 'address': 'Metropolis'}

{'name': 'Bruce Wayne', 'age': 'Unknown', 'address': 'Gotham'}

NESTED DICTIONARIES

```
01 # Nested Dictionaries
02 contact = {"Batman": {
03             "name": "Bruce Wayne",
04             "age" : "Unknown",
05             "address": "Gotham",
06             "sidekick": {"Robin": {
07                           "name": "Richard Grayson",
08                           "age" : 16,
09                           "address": "Bludhaven"
10                         }
11             }
12         }
13     }
14 # Resolve each key in precedence from left to right like lists
15 print(contact["Batman"]["sidekick"]) # {'Robin': {'name': 'Richard Grayson', 'age': 16, 'address': 'Bludhaven'}}
16 print(contact["Batman"]["sidekick"]["Robin"]["name"]) #Richard Grayson
```

ADDING OR MODIFYING ELEMENTS

- Change the value associated with a particular key
 - Use the [] square brackets
- Add a new key-value pair by assigning a value to a new key
 - Use the square brackets [] with existing key
- Removing a key-value pair
 - Use del or .pop

```
1  # .pop method on dictionary
2  hero_1 = {"name" : "Clark Kent",
3           "age" : 33,
4           "address" : "Metropolis",
5           "email" : "ckent@dailyplanet.com"}
6  age = hero_1.pop("age") #get the value for age and remove key:value pair
7  print(age) # 33
8  del hero_1["address"]
9  print(hero_1) # {'name': 'Clark Kent', 'email': 'ckent@dailyplanet.com'}
```

A NOTE ON .POP

- It returns the value associated with the provided key (it also removes the key-value pair)
- You can provide an optional default which is returned in the case the key does not exist
 - If no default is provided then None is returned

```
1 hero_2 = { "name" : "Bruce Wayne",  
2           "age" : "Unknown",  
3           "address" : "Gotham" }  
4 email = hero_2.pop("email", "Unknown")  
5 print(email) #Unknown  
6 print(hero_2) #{'name': 'Bruce Wayne', 'age': 'Unknown', 'address': 'Gotham'}
```


SELF-CHECK

- As you are likely aware: Superman:The Movie with Chris Reeves and Margot Kidder was released in 1978
- Update 1971 to 1978
- Change the release date for Superman: The Movie from 1971 to 1978

```
01  # Self check
02  # nested dictionary
03  films = {"Superman Movies" : { "Superman: The Movie" :
04                                  {"genre" : "Action",
05                                  "release" : 1971,
06                                  "income" : 300500000 },
07                                  "Superman II" :
08                                  {"genre" : "Action",
09                                  "release" : 1980,
10                                  "income" : 216000000 },
11                                  "Superman III" :
12                                  {"genre" : "Comedy",
13                                  "release" : 1983,
14                                  "income" : 80250000 },
15                                  "Superman IV" :
16                                  {"genre" : "Tragedy",
17                                  "release" : 1987,
18                                  "income" : 36700000}}
19      }
```

```
films["Superman Movies"]["Superman: The Movie"]["release"] = 1978
```

DICTIONARY METHODS

- `.keys()` returns a view object of all keys in the dictionary
 - `.values()` returns a view object of all values in the dictionary
 - `.items()` returns a view object of tuples of (key, value) pairs
-
- View objects are similar to lists but they are read-only
 - i.e., they cannot be modified
 - But they are objects and so they reference the actual underlying data (not a copy)
 - Support iteration, membership checks, etc

KEYS

```
01 my_dict = {'a': 1, 'b': 2, 'c': 3}
02 # Create a view object for keys
03 keys_view = my_dict.keys()
04 # keys_view can "see" all of the keys
05 print(keys_view) #dict_keys(['a', 'b', 'c'])
06 print(f"Initial keys view: {keys_view}")
07 # Initial keys view: dict_keys(['a', 'b', 'c'])
08 # Modify the dictionary
09 my_dict['d'] = 4
10 # After modifying the dictionary, the keys_view variable
11 print(f"Keys view after dictionary modification: {keys_view}")
12 # Keys view after dictionary modification: dict_keys(['a', 'b', 'c', 'd'])
```


VALUES

- 1 *# Same thing with .values()*
 - 2 `values_view = my_dict.values()`
 - 3 `print(values_view)`
 - 4 `my_dict['e'] = 5`
 - 5 `print(f"Values view after dictionary modification: {values_view}")`
- Values work pretty much the same as keys – you have a view over them, not accessing the actual values:
“Values view after dictionary modification: dict_values([1, 2, 3, 4, 5])”

ITEMS

```
1  # .items() returns an 'iterable'  
2  # Iterable - means it is a data structure that can be iterated over  
3  items_view = my_dict.items()  
4  print(f"Items view (tuples): {items_view}")
```

- Lists and dictionaries are both objects that are iterable
 - That means that they have been defined in a way where you can process them in an order that makes sense
 - You can provide a method (named anything but usually `items()`) that can return an iterable even if your data structure is very complicated
 - Particularly important down the road for trees

COMMON ITERATION PATTERNS

- Iterating over lists comes in many common patterns that you can use in your code:
 - Iterate over keys
 - Iterate over values
 - Iterate over tuples
 - Iterate over keys but access values

```
01 # Iterating over dictionaries
02 my_dict = {'a': 1, 'c': 3, 'b': 2}
03 my_dict['d'] = 4
04 #Iterates over the keys
05  #(in the order they were inserted)
06 for k in my_dict:
07     print(k, end=" ")
08 print() # a c b d
09 #Iterates over the keys
10 #Retrieve keys view
11 for k in my_dict.keys():
12     print(k, end=" ")
13 print() # a c b d
14 #Iterates over the values
15 for v in my_dict.values():
16     print(v, end=" ")
17 print() # 1 3 2 4
18 # iterable - data structure that can be iterated over
19 # .items() - returns an iterable of tuples
20 # Iterates over tuples
21 for k,v in my_dict.items():
22     print(k,v, end=" ")
23 print() # a 1 c 3 b 2 d 4
24 for item in my_dict.items():
25     print(item[0], item[1], end=" ")
26 print() # a 1 c 3 b 2 d 4
27 #Iterate over the keys but access the values
28 for k in my_dict:
29     print(k, my_dict[k], end=" ")
30 print() # a 1 c 3 b 2 d 4
```

.GET

- `.get(key, default=None)` returns the value associated with a key (or default) if the key doesn't exist

`.get` is useful because of the ability to supply a default. The `[]` operator does not have a default. Therefore extra care is required when using `[]` to avoid a `KeyError` which may crash your program

WHEN TO USE DICTIONARIES

- Need to create an association
- Need fast lookups based on a specific identifier (key)
- Order of items doesn't matter
- Examples:
 - Configuration settings
 - Counting frequencies
 - Representing objects or records

FIND A PARTNER: CIPHERS

- You are given a list that represents a cipher.
 - Even positions contain symbols
 - Odd positions contain meanings
- Assume one UNICODE symbol for each ASCII capital letter character
- Task: Your task is to translate this list into a dictionary so it can be used to decode messages.
- Task: Create a function called `transform_cipher` that takes in a list and returns a dictionary

FIND A PARTNER: GENERALIZATION

- Assume that we do not want anyone to know how we have implemented the cipher – it could be a list, or it could be a dictionary
- Write two functions:
 - One named encode which takes in a unencoded string, and returns an encoded string of symbols – but it should use a list inside for doing the encoding
 - One named decode which takes in an encoded string of symbols and returns the decoded string – but it should use a dictionary inside for doing the decoding

FIND A PARTNER: CHARACTER FREQUENCY

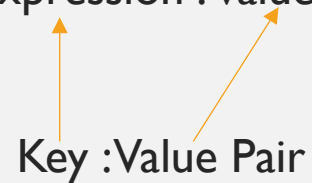
- Consider that we want to count the number of times a letter appears in any text
- Example: "the quick brown fox jumps over the lazy dog"
 - {'b':1, 'r':1, 'o':2, 'w':1, 'n':1, 'd':1, 'g':1}
- Write an algorithm that creates this dictionary from any text
 - What questions do you need to ask about the design?
- Now, assume that we want to reuse this algorithm again and again – what is the function that you would define?

DICTIONARY COMPREHENSIONS

- Convert a sequence into a dictionary

```
new_dict = {key_expression : value_expression for item in iterable}
```

Key :Value Pair



Example:

```
numbers = [1, 2, 3, 4]
```

```
squared_dict = {num: num ** 2 for num in numbers}
```

Result: {1: 1, 2: 4, 3: 9, 4: 16}

EXAMPLES

#optional condition

```
original_dict = {'a': 10, 'b': 25, 'c': 5, 'd': 30}
```

```
filtered_and_doubled = {k: v * 2 for k, v in original_dict.items() if v > 15}
```

Result: {'b': 50, 'd': 60}

dictionary from list of tuples

```
data = [('apple', 1), ('banana', 2), ('cherry', 3)]
```

```
fruit_counts = {fruit: count for fruit, count in data}
```

Result: {'apple': 1, 'banana': 2, 'cherry': 3}

```
print(fruit_counts)
```

CONDITIONAL EXPRESSION

- Can be used anywhere an expression is expected
value_if_true if condition else value_if_false

Examples:

conditional expression

time = 13

print("good morning" if time < 12 else "good afternoon")

#if-else condition

data = {'A': 10, 'B': 5, 'C': 12}

transformed_values = {k: "High" if v > 10 else "Low" for k, v in data.items()}

Result: {'A': 'High', 'B': 'Low', 'C': 'High'}

SELF-CHECK

What is stored in status?

Self-check

age = 20

status = "Adult" if age >= 18 else "Minor"

SELF-CHECK

```
realnames = { "Marilyn Monroe" : "Norma Mortenson",  
              "Nicolas Cage"   : "Nicolas Coppola",  
              "Albert Brooks" : "Albert Einstein",  
              "Elton John"    : "Reginald Dwight" }
```

#What is stored in rev?

```
rev = { realnames[k]:k for k in realnames }
```

```
print(rev)
```

EXTRA
PRACTICE WITH
MUTABLE
OBJECTS

```
mt1 = [99, 98, 97, 90, 82, 55]
```

```
mt2 = [96, 94, 90, 81, 47]
```

```
grades = { "MT1" : mt1,  
           "MT2" : mt2 }
```

#also alters the mt1 list

```
grades["MT1"].append(46)
```

#changes what grades["MT1"] refers to

```
grades["MT1"] = grades["MT1"][0:2]
```

```
print(grades)
```

```
print(mt1)
```

FIND A PARTNER: FOOD CHAIN

- **Find a Partner: Food Chain**

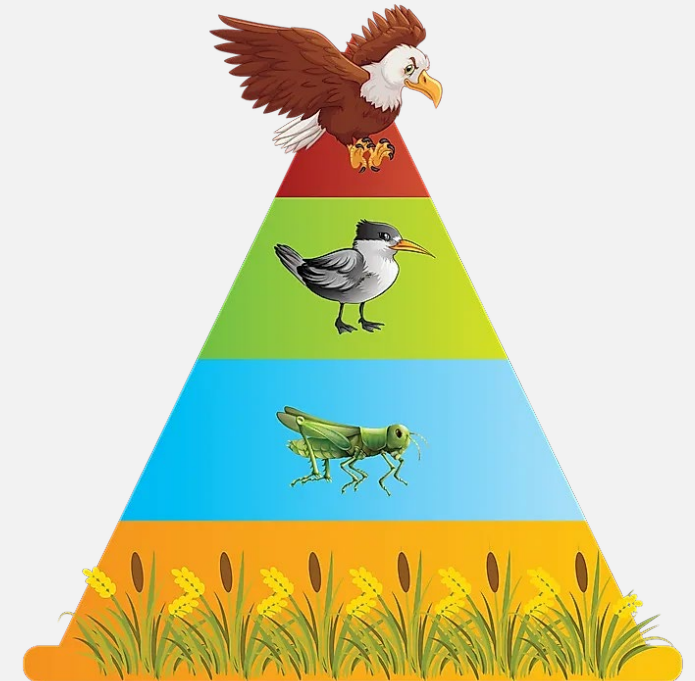
- Some creatures producers, meaning they do not eat anyone other creature. Others are apex predators meaning they are eaten by no one else. Some creatures only eat producers, meaning they are vegetarian.
- Assume that we have food systems that are stored in lists.
- The first in the list is the predator and the remaining is a list of prey that they eat.
- Example: Birds eat Prawn, Mussels, Crab, Limpets and Whelk

- **Consider:**

- How would we transform these into a dictionary that would help us manage our data better?
- What is the design of your dictionary? Do you need to know anything else?
- How do we do it as a loop? How do we do it as a comprehension?

- **After creating that dictionary, answer each of the following questions?**

- Determine the apex predators
- Determine the producers
- Determine the tastiest organisms (most eaten)
- Determine the omnivores who eat both producers and non-producers?
- At each step consider - can you use parts of your previous solution?



FIND A PARTNER

- Consider the predator-prey relationship: predators eat prey

Bird, Prawn, Mussels, Crab, Limpets, Whelk

Crab, Mussels, Limpets

Fish, Prawn

Limpets, Seaweed

Lobster, Crab, Mussels, Limpets, Whelk

Mussels, Phytoplankton, Zooplankton

Prawn, Zooplankton

Whelk, Limpets, Mussels

Zooplankton, Phytoplankton

TRY AT HOME

- Create a dictionary to store information about your favorite book
 - Author, title, year, pages, etc
- Create a dictionary where the keys are book titles and the values are a dictionary like the one you created above
 - Add 3 books to this dictionary
 - update the pages for each entry to increase it by 1
- For a given string create a dictionary to count the occurrences of each letter in the string

CONCLUSION

- Dictionaries are a data structure capable of storing key and value pairs
- They offer fast lookup capabilities given a key
- They are a mutable object
- But the keys must be comprised of an immutable type
- Combined with Lists they offer mechanism to represent many data storage options